DOSSIER DE PROJET DWWM 2026

Paul Bourlier

L'Empreinte

I. INTRODUCTION & REMERCIEMENTS **3**
II. COMPÉTENCES DU RÉFÉRENTIEL COUVERTES PAR LE PROJET **5**
III. PRÉSENTATION DU PROJET **6**
3.1 Contexte et Origine (Sandrine Tribhou) 6
3.2 La Mission : Créer la "Synchronicité" 6
3.3 Objectifs du Projet 6
3.4 Périmètre et limites du projet (Le MVP) 7
IV. ANALYSE ET CONCEPTION **8**
4.1 Analyse Fonctionnelle (Use Case) **8**
4.2 Spécifications (User Stories de 1 à 15) **10**
4.3 Modélisation des données (MCD, MLD, MPD) **11**
4.3.1 Approche Conceptuelle (MCD) 11
4.4 La phase de Grisaille (Wireframing) 13
Focus sur l'architecture du pied de page : 16
V. ENVIRONNEMENT TECHNIQUE **16**
VI. RÉALISATION FRONT-END **17**
6.1 Identité Visuelle et Processus Créatif (Figma) **17**
6.2 Intégration Tailwind CSS v4 et Twig : La légèreté du geste **21**
6.3 Architecture Twig et AssetMapper 22
6.4 Gestion du Design System et Réutilisabilité 23
6.5 Éthique Numérique : Accessibilité et Éco-conception 24
7.1 Architecture, Migrations et Sécurité **25**
7.1.1 Intégrité du Schéma et Mapping ORM 25
7.1.2 Anonymisation et Confidentialité (UUID) 26
7.1.3 Sécurité et Authorization (ROLE_ADMIN) 26
7.1.4 Gestion des erreurs de privilèges (Subscriber) 27
7.1.5 Le contrôle des informations saisies 27
7.1.6 Protection contre les failles majeures (XSS et CSRF) 28
7.2 Logique Métier : L'intelligence du Sanctuaire **29**
7.2.1 La Transmission du Savoir (LexiconLinker) 29
7.2.2 La Maîtrise du Temps (Algorithme de Réservation) 31
7.2.3 La Sécurisation des données sensibles 32
7.2.4 Indexation et Recherche Croisée 33
7.3 Validation Fonctionnelle (Tests réels) 35
7.4 Gestion de la Relation Patient (CRM) 36
7.4.1 Historisation et Déperdition de sens 36
7.4.2 Le Dialogue Bilatéral : L'Écho de la praticienne 38
7.5 : L'Économie du Soin — Le Wallet Numérique 39
7.6 Ma méthode pour vérifier le bon fonctionnement 40
7.7 Ma démarche de veille : Apprendre et sécuriser 41
7.8 Déploiement et mise en production 41

7.8.1 Procédure technique de mise en production 42
7.9 La Dualité Narrative : Sanctuaire et Blog 43
7.9.1 Le Sanctuaire — L'intimité de la pratique 43
7.9.2 Le Blog — La sagesse en partage 44
7.10 Les Itinéraires de Soin : La réponse aux maux du quotidien 45
7.10.1 La Libération du Mental (Shiatsu Crânien) 45
7.10.2 L'Ancrage du Hara (Harmonie Digestive) 45
7.10.3 Le Geste Originel (Shiatsu Traditionnel Namikoshi) 45
7.11 La Dynamique Éditoriale : Le Cycle des Saisons (Espace Blog) 46
7.12 L'adaptation mobile : Le Sanctuaire au creux de la main 47
7.12.1 Le défi de la "Chicane" sur petit écran 49
7.12.2 Une approche technique "Mobile-First" avec Tailwind v4 49
8.1 Bilan Personnel et Pédagogique 51
8.2 Évolutions futures du projet 51
IX. CONCLUSION 52
ANNEXE 1 : Carnet de bord des fonctionnalités (User Stories) 53
User Stories 53
ANNEXE 2 : Sécurité et protection des données (RGPD) 55
2.1 La politique de confidentialité par défaut (Privacy by Design) 55
2.2 La gestion du "Droit à l'oubli" 55
2.3 Sécurisation des accès et transparence 55

I. INTRODUCTION & REMERCIEMENTS

Ce projet est né de la rencontre entre un métier millénaire, le Shiatsu, et les technologies les plus modernes du développement Web. Passionné par l'alliance entre le sens et la technique, j'ai choisi de consacrer mon projet de fin d'année à la création de **"L'EMPREINTE"**, un écosystème numérique dédié au cabinet de Sandrine Tribhou (Toulouse/Tournefeuille).

L'application n'est pas un simple site vitrine ; elle a été conçue comme un prolongement numérique du cabinet, un sanctuaire de savoir et de soin où la technologie s'efface derrière une expérience utilisateur épurée. Mon défi a été de répondre aux standards de la "Haute Couture" numérique : une interface invitant au calme, une sécurité maximale pour les données de santé (UUID, hachage) et une architecture logicielle robuste sous Symfony 8.

L'enjeu majeur de cette réalisation réside dans l'autonomie totale offerte à la praticienne : via un espace de pilotage dédié, elle peut orchestrer son catalogue, son lexique et ses publications sans aucune barrière technique, garantissant la pérennité de son empreinte numérique.

Ce dossier retrace mon cheminement complet, de l'analyse fonctionnelle à la mise en œuvre technique. Il représente l'aboutissement de ma formation, matérialisant ma capacité à transformer une vision thérapeutique en un outil métier performant et évolutif.

Remerciements :

Je tiens à exprimer ma profonde gratitude à :

- **Sandrine Tribhou**, pour m'avoir confié l'image de son cabinet et pour sa confiance précieuse dans la définition de ses besoins métier.

- **Mon enseignant**, pour sa guidance technique et son exigence qui m'ont poussé à la rigueur nécessaire au développement sous Symfony.

- **Mes collègues de promotion**, pour la richesse de nos échanges tout au long de ce parcours de formation.

II. COMPÉTENCES DU RÉFÉRENTIEL COUVERTES PAR LE PROJET

Le projet **L'EMPREINTE** a été conçu pour valider l'ensemble des compétences techniques attendues pour le Titre Professionnel de Développeur Web et Web Mobile (DWWM).

Compétence Professionnelle (CP)	Mise en œuvre dans le projet L'EMPREINTE
ACTIVITÉ TYPE 1	Développer la partie front-end d'une application web sécurisée
CP 1. Installer et configurer son environnement de travail	Mise en place d'une architecture conteneurisée sous **Docker** (**FrankenPHP**, PostgreSQL 14). Gestion des versions via Git/GitHub.
CP 2. Maquetter une application web ou web mobile	Conception de 21 frames mobiles et 7 frames Desktop sous **Figma**. Processus itératif : US > wireframes > maquettes haute définition.
CP 3. Réaliser une interface utilisateur statique	Intégration des maquettes via le moteur de template **Twig** et le framework **Tailwind CSS v4** (moteur de rendu Rust).
CP 4. Développer une interface utilisateur dynamique	Utilisation de **Stimulus JS** pour l'interactivité (Popovers dynamiques du lexique) et gestion des messages flash.
ACTIVITÉ TYPE 2	Développer la partie back-end d'une application web sécurisée
CP 5. Mettre en place une base de données relationnelle	Modélisation et implémentation de 10 tables sous **PostgreSQL 14**. Utilisation rigoureuse des types **Decimal** (Wallet) et **UUID**.
CP 6. Développer des composants d'accès aux données	Implémentation de l'ORM **Doctrine**. Utilisation des **Repositories** pour la récupération optimisée des soins et du moteur de recherche.
CP 7. Développer des composants métier côté serveur	Création du **LexiconLinkerService** (traitement de texte) et de l'algorithme de réservation avec gestion des marges de **"Respiration"**.

CP 8. Documenter le déploiement d'une application	Rédaction de la procédure de mise en production automatisée via **SSH** et **Docker**, documentée dans le README du projet.

III. PRÉSENTATION DU PROJET

3.1 Contexte et Origine (Sandrine Tribhou)

Le projet "L'EMPREINTE" naît de la volonté de Sandrine Tribhou, praticienne en Shiatsu Traditionnel Namikoshi et Médecine Traditionnelle Chinoise (MTC).

Le constat de départ était clair : le cabinet disposait d'une présence web qui ne reflétait plus l'excellence et la sérénité de l'approche thérapeutique proposée. Il était nécessaire d'évoluer vers un écosystème numérique alliant rigueur professionnelle et élégance sobre. L'enjeu est de traduire visuellement le "souffle" (le Qi) et la précision du geste Namikoshi à travers une interface privilégiant une esthétique épurée. Pour la praticienne, l'objectif est d'atteindre une autonomie totale (gestion du catalogue, du lexique, des articles) sans obstacle technique.

3.2 La Mission : Créer la "Synchronicité"

L'application engage le patient dans un parcours fluide dont l'acte thérapeutique commence dès les premières communications. Chaque étape (lecture d'un article, compréhension d'un terme technique via le lexique, échange de messages) converge naturellement vers l'acte de soin physique. C'est ce que j'ai nommé le **Flux de Synchronicité**.

3.3 Objectifs du Projet

Pour répondre aux besoins de la praticienne et de ses patients, quatre objectifs majeurs ont été fixés :

1. **Architecture de pointe** : Développement d'une base de données relationnelle robuste (PostgreSQL) capable de gérer l'historique complet de la relation patient.

2. **Transmission du Savoir** : Automatisation de la pédagogie grâce à un "Sanctuaire" (espace de savoir) et un Lexique dynamique, rendant la MTC accessible sans rompre le flux de lecture.
3. **Gestion Métier Exclusive** : Intégration d'outils transactionnels innovants, tels que le Wallet (portefeuille numérique) et la génération de cartes cadeaux sécurisées.
4. **Expérience Utilisateur "Zen"** : Utilisation de la puissance de Tailwind v4 pour créer une interface ultra-rapide où le vide graphique sert de respiration au visiteur.

3.4 Périmètre et limites du projet (Le MVP)

Pour répondre à l'échéance de ma formation tout en garantissant le meilleur niveau de qualité, j'ai dû définir un périmètre précis, appelé MVP (Produit Minimum Viable). Il est important de préciser ce que cette version de "L'Empreinte" ne contient pas encore :

Paiement en ligne réel : Bien que la structure du Wallet soit prête et le type Decimal configuré en base de données, l'intégration de l'API Stripe est simulée pour cette présentation. Le passage en production nécessiterait une phase de test supplémentaire avec des clés API réelles.

Gestion multi-praticiens : L'application est strictement dédiée à l'activité de Sandrine Tribhou. La base de données n'est pas conçue pour gérer plusieurs agendas de thérapeutes différents.

Automatisation des mails : Pour éviter les problématiques de serveurs de mails (Spam) lors de la soutenance, les confirmations de rendez-vous sont gérées en interne dans l'espace patient ("Synchronicité") plutôt que par l'envoi de courriels externes.

Tests Automatisés : Pour cette version initiale, j'ai privilégié les tests manuels et la validation rigoureuse des scénarios d'usage. L'implémentation de tests unitaires (PHPUnit) et fonctionnels est prévue pour la phase suivante du projet, afin d'assurer la non-régression lors de l'ajout de nouvelles fonctionnalités.

Reconnaître ces limites me permet de garantir que chaque fonctionnalité présente est stable, sécurisée et répond parfaitement aux besoins prioritaires de la praticienne.

IV. ANALYSE ET CONCEPTION

4.1 Analyse Fonctionnelle (Use Case)

Pour concevoir une interface qui respecte la fluidité du parcours de soin, j'ai réalisé un Diagramme de Cas d'Utilisation (UML). Ce travail d'analyse a permis d'identifier précisément les interactions entre les trois types d'acteurs de l'écosystème :

L'Internaute (Visiteur) : Il est en phase de découverte. Il consulte l'approche de Sandrine, parcourt le catalogue des soins et s'informe via le Sanctuaire.

Le Patient (Utilisateur authentifié) : Il est engagé dans une démarche de soin. Il dispose d'un espace personnel pour dialoguer avec la thérapeute (Synchronicité) et peut transformer son intention en acte en prenant rendez-vous via l'agenda.

L'Administratrice (Sandrine) : Elle pilote l'ensemble. Elle gère les contenus pédagogiques, répond aux messages et assure la gestion du cabinet (soins et lexique).

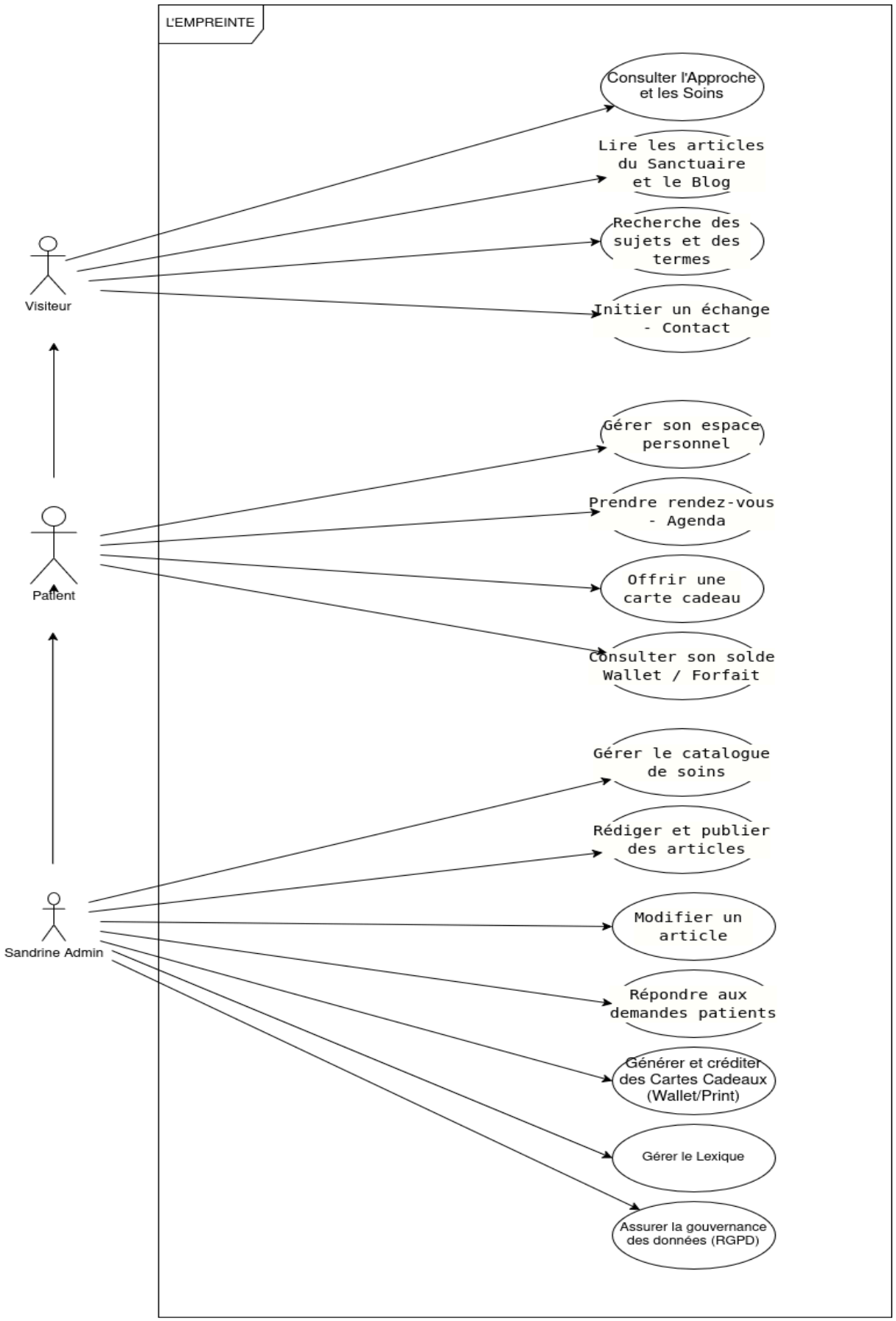


Diagramme de Cas d'Utilisation illustrant le cycle de vie du patient, de la découverte à l'engagement.

4.2 Spécifications (User Stories de 1 à 15)

N°	User Story (Rôle / Intention / But)	Critères d'Acceptation
US 1	**Visiteur** : Consulter le catalogue des soins pour choisir une prestation adaptée.	Grille 3 colonnes, tarifs Decimal, durées précises, design Zen.
US 2	**Visiteur** : Initier un échange thérapeutique pour exprimer mon besoin	Formulaire sécurisé, historisation en BDD, création automatique du compte.
US 3	**Patient** : Réserver un créneau en temps réel via un calendrier intuitif.	Calendrier interactif, pas de double réservation, gestion de la "Respiration" préservant la qualité de l'accueil entre deux séances.
US 4	**Utilisateur** : S'authentifier pour accéder à son espace sécurisé.	Login avec hachage (BCRYPT), protection CSRF, redirection selon le rôle.
US 5	**Patient** : Suivre son parcours de soin et l'historique des échanges.	Espace personnel listant les messages et les rendez-vous passés/futurs.
US 6	**Visiteur** : Effectuer une recherche globale pour trouver un article ou un mot technique.	Barre de recherche active, résultats filtrés par pertinence (Blog/Lexique).
US 7	**Client** : Sélectionner et acheter une "Carte Expérience" pour un proche.	Choix du montant, formulaire de personnalisation (Nom, message), paiement Stripe.
US 8	**Acheteur** : Recevoir la Carte Cadeau par email avec son code unique.	Génération d'un code haché, envoi automatique d'un PDF raffiné et imprimable.
US 9	**Bénéficiaire** : Utiliser mon code cadeau pour réserver un soin.	Champ de saisie du code sur le calendrier, déduction du prix, validation sans paiement.
US 10	**Admin** : Accéder à un Dashboard de pilotage centralisé.	Vue d'ensemble des RDV, solde du cabinet, accès rapide aux outils de gestion.
US 11	**Admin** : Gérer les dossiers patients et l'historique thérapeutique.	Liste des patients, accès aux UUID sécurisés, consultation des échanges passés.
US 12	**Admin** : Configurer le planning et les temps de "Respiration".	Réglage des jours d'ouverture, ajout automatique de battements entre deux soins.
US 13	**Admin** : Gérer le catalogue des prestations (CRUD Soins).	Ajouter, modifier les tarifs ou désactiver une prestation en temps réel.
US 14	**Admin** : Gérer le dictionnaire technique (CRUD Lexique).	Créer de nouvelles définitions qui s'injecteront automatiquement dans les articles.
US 15	**Admin** : Assurer la conformité RGPD et la gouvernance des données.	Gouvernance des données sensibles : Anonymisation structurelle par UUID et respect du droit à l'oubli.

4.3 Modélisation des données (MCD, MLD, MPD)

La structure de données de "L'Empreinte" a été modélisée pour traduire numériquement la fluidité de la relation thérapeutique. J'ai conçu un schéma relationnel de 10 tables sous PostgreSQL, articulé autour de quatre pôles majeurs : l'Identité, l'Expertise, la Synchronicité (échanges et rendez-vous) et l'Économie du soin.

4.3.1 Approche Conceptuelle (MCD)

Avant de définir les tables techniques, j'ai modélisé les interactions logiques du cabinet de Shiatsu. Le Modèle Conceptuel de Données repose sur les règles de gestion suivantes :

Le Patient (User) est au cœur du système. Il peut **initier** plusieurs échanges thérapeutiques et **effectuer** plusieurs paiements.

Le Rendez-vous (Appointment) fait le lien entre un Patient et un **Soin (Treatment)** à une date précise.

Le Savoir (Article) peut citer plusieurs termes du **Lexique**, et un terme peut apparaître dans plusieurs articles (Relation n,n).

La Carte Cadeau (GiftCard) est achetée par un Patient pour offrir un Soin spécifique à un bénéficiaire.

Cette étape m'a permis de valider la cohérence des relations avant de passer à l'implémentation physique sous PostgreSQL.

Du concept à la réalisation technique

Une fois cette structure logique établie, j'ai traduit ces relations en un schéma technique exploitable par le système de gestion de base de données. Ce passage du conceptuel (le "quoi") au physique (le "comment") est une étape cruciale : elle permet de s'assurer que chaque règle métier, comme l'impossibilité de supprimer un patient ayant des rendez-vous à venir, est protégée par des contraintes d'intégrité solides au cœur même de PostgreSQL.



Modèle Physique de Données (MPD) illustrant l'implémentation rigoureuse des types et des relations sous PostgreSQL 14.

A. Le Pôle Identité et Sécurité (Table User)

L'entité **User** est le pivot central de l'écosystème. Contrairement à une structure classique, elle

intègre nativement la gestion du **Wallet** (type Decimal pour la précision monétaire) et utilise des identifiants **UUID**. Ce choix technique garantit que l'identité d'un patient n'est jamais exposée de manière prédictible dans les URLs, assurant une confidentialité optimale dès la racine du système.

B. Le Pôle Expertise et Savoir (Tables Treatment, Lexicon, Article)

Ce pôle matérialise le savoir-faire de la praticienne. La table **Lexicon** est conçue pour être transverse : elle n'est pas une simple liste isolée, mais une ressource dynamique dont les données sont injectées en temps réel dans les contenus de la table **Article**. Cette interconnexion permet une transmission pédagogique fluide, pilier de l'expérience utilisateur dans le Sanctuaire.

C. Le Pôle Synchronicité et Engagement (Tables TherapeuticExchange, Appointment)

L'innovation métier de "L'Empreinte" repose sur le lien entre l'échange écrit et l'acte de soin. Chaque **TherapeuticExchange** est historisé et rattaché à l'utilisateur, permettant de conserver l'intention du patient. La table **Appointment** vient clore ce cycle en liant le soin choisi, le patient et le créneau temporel, tout en respectant les contraintes de "Respiration" calculées par le système.

D. Le pôle Économique (Tables, Payment, GiftCard)

Ce dernier pilier sécurise les flux financiers au sein de l'écosystème. L'utilisation de la table **Payment** permet une traçabilité totale des transactions, tandis que l'entité **GiftCard** offre une flexibilité de gestion pour les bons cadeaux. D'un point de vue technique, le choix du type **Decimal(10,2)** pour les montants financiers garantit une précision bancaire absolue, évitant les erreurs d'arrondi inhérentes aux types flottants classiques.

4.4 La phase de Grisaille (Wireframing)

Avant d'appliquer l'identité visuelle 'Zen-Sand', une étape de prototypage basse fidélité, ou 'Grisaille', a été rigoureusement respectée. L'objectif premier est de valider la hiérarchie des informations et le parcours de navigation sans être distrait par l'esthétique. Cette phase permet de s'assurer que chaque élément (Boutons d'appel à l'action, sélecteurs, blocs de texte) répond précisément aux besoins fonctionnels définis dans les User Stories.



L'étape de grisaille m'a notamment permis de valider le concept de 'Chicane' visuelle. En évacuant la séduction de la typographie et des images, j'ai pu tester l'équilibre des masses au pixel près. Cette méthode garantit que le parcours de lecture reste intuitif, assurant une transition fluide entre l'accroche philosophique et l'appel à l'action.



La phase de wireframing de la page de réservation a permis de valider la 'Chicane' entre le choix du soin et la sélection du créneau, garantissant une navigation sans friction avant l'application de la charte graphique.

Focus sur l'architecture du pied de page :

Pour le footer, j'ai défini que l'équilibre visuel repose sur trois blocs verticaux de densité équivalente, assurant une base stable à l'application.

V. ENVIRONNEMENT TECHNIQUE

Pour développer ce projet, j'ai utilisé **Visual Studio Code**. Pour garder un code propre et bien rangé, j'ai installé **PHP-CS-Fixer**. Cela me permet de formater mes fichiers automatiquement avec le raccourci Ctrl+Maj+I. C'est indispensable pour que le code soit facile à lire pour n'importe quel développeur.

Le suivi du projet

Pour ne pas perdre mon travail et suivre chaque étape, j'utilise **Git** et **GitHub**. J'ai travaillé avec un système de "branches" : j'ai créé une branche spéciale (comme feature/sanctuary-logic) pour chaque nouvelle fonctionnalité. Une fois que tout fonctionne bien, je fusionne cette branche sur mon projet principal.

Infrastructure Docker & FrankenPHP

Le choix de Docker garantit la reproductibilité totale de l'environnement entre le poste de développement et la production. J'ai configuré trois conteneurs distincts :

1. **Conteneur PHP** : Utilisant l'image officielle PHP 8.4-zts.

2. **Conteneur Database** : PostgreSQL 14 pour sa gestion native des UUID.

3. **Conteneur pgAdmin** : Interface d'administration pour la surveillance des données.

L'avantage FrankenPHP

Contrairement à un serveur classique (Nginx/Apache), FrankenPHP permet une montée en charge fluide et des temps de réponse divisés par deux, essentiels pour préserver le calme et la fluidité de la navigation.

VI. RÉALISATION FRONT-END

6.1 Identité Visuelle et Processus Créatif (Figma)

Au lieu d'utiliser des modèles tout faits, j'ai conçu un design sur-mesure que j'appelle la **"Chicane"**. C'est une alternance asymétrique entre texte et image. Ce n'est pas juste joli : ça oblige l'œil à ralentir. Sur le site de Sandrine, on ne "scrolle" pas, on déambule. C'est ma façon d'apporter de la sérénité dès les premières secondes de navigation.

A. La Genèse Sémantique : Le Choix de l'Épure

La première étape a consisté à traduire les valeurs de la Médecine Traditionnelle Chinoise en codes graphiques modernes. Le choix de la palette **Zen-Sand (#F7F1E8)** répond à un impératif de neutralité, offrant au regard un repos immédiat. L'utilisation du vert **Zen-Forest (#8A9A5B)** pour les éléments d'interaction symbolise la vitalité et l'équilibre organique. Cette étape a permis de poser les bases de la charte graphique avant toute mise en page.

B. L'Architecture de l'Information (Wireframing)

Avant de manipuler les couleurs et les images, j'ai élaboré des maquettes basse fidélité, des "wireframe", ou de **"Grisaille"**. Cette phase de wireframing est cruciale : elle permet d'éprouver la hiérarchie des masses et de valider notamment le concept de **"Chicane" visuelle**. En travaillant uniquement sur les échelles de gris, j'ai pu m'assurer que le parcours du patient reste fluide et que les appels à l'action (CTA) sont positionnés de manière stratégique sans jamais créer de stress visuel.

C. Conception Modulaire (Atomic Design)

Pour assurer l'autonomie de la praticienne et la maintenabilité du code, j'ai construit le projet sur Figma via des **Composants Maîtres**. Chaque élément (Header, Footer, Boutons, Cartes de soin) a été conçu comme un atome indépendant. Cette rigueur garantit une cohérence absolue entre les 28 frames du projet (Desktop et Mobile) : toute modification sur le composant maître se répercute instantanément sur l'ensemble de l'écosystème.

D. Le Rendu Haute Fidélité et l'Expérience Utilisateur

La phase finale a consisté à intégrer les polices de caractères "Livre d'Art" (**Cormorant** et **EB Garamond**) et les visuels de soin. Le résultat est une interface responsive où le vide graphique est utilisé comme une respiration nécessaire, transformant la navigation technique en un véritable

cheminement vers le bien-être.



Wireframe basse fidélité validant l'équilibre des masses en chicane.



Rendu final : l'interface de réservation adopte une structure asymétrique en chicane pour rompre avec la rigidité des formulaires classiques, invitant le patient à un cheminement apaisé intégrant la charte Zen-Sand et la typographie éditoriale.

6.2 Intégration Tailwind CSS v4 et Twig : La légèreté du geste

Pour l'intégration, j'ai choisi **Tailwind CSS v4** associé au moteur **Twig**.

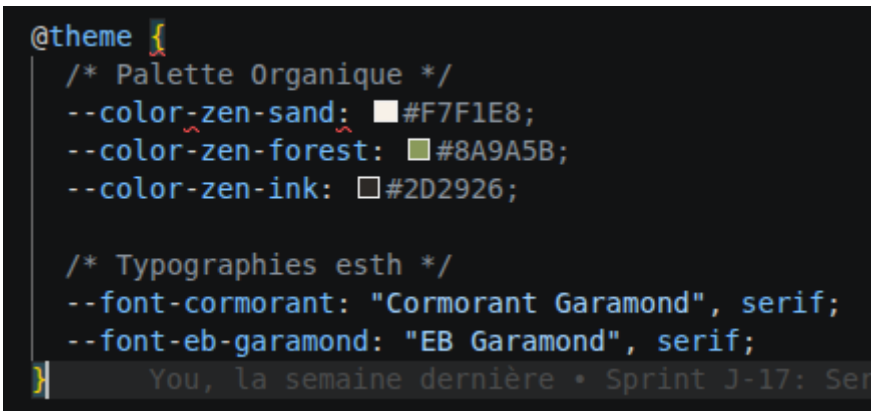
Pourquoi ce choix ?

- **La performance :** Tailwind v4 utilise un moteur en Rust extrêmement rapide. Cela garantit un affichage instantané, essentiel pour ne pas rompre la sérénité du visiteur.
- **Le respect du DRY (Don't Repeat Yourself) :** Grâce à l'héritage de template de Twig, j'ai centralisé le Header et le Footer dans un fichier maître (base.html.twig). Les pages de soins ou du lexique ne contiennent que leur contenu spécifique, ce qui simplifie énormément la maintenance.

L'approche par "Utilitaires"
Plutôt que d'écrire des fichiers CSS interminables, j'utilise les classes de Tailwind directement dans l'HTML. Cela me permet de garder un code très léger (poids plume) et de voir immédiatement le résultat visuel sans changer de fichier.

Configuration de l'ADN Visuel

J'ai configuré un thème personnalisé (@theme) pour injecter les variables de la charte "L'EMPREINTE" directement dans le moteur de rendu. Ainsi, les couleurs zen-sand, zen-forest et les typographies cormorant ou eb-garamond sont devenues des outils standards du projet.



Injection des variables de la charte graphique dans le moteur Tailwind v4.

6.3 Architecture Twig et AssetMapper

Pour la gestion des fichiers CSS et JavaScript, j'ai fait le choix de la modernité avec **AssetMapper**. Contrairement aux anciennes méthodes qui demandaient d'installer des usines à gaz comme Node.js ou Webpack, AssetMapper permet de charger les ressources directement.

C'est une approche beaucoup plus légère qui colle parfaitement à l'esprit "épuré" de L'Empreinte. Le navigateur ne télécharge que ce dont il a besoin, quand il en a besoin, ce qui garantit un affichage instantané des polices **Cormorant** et **EB Garamond**.

Ce choix d'**AssetMapper** participe à une démarche d'**éco-conception**. En évitant l'installation de Node.js et de processus de compilation lourds, je réduis l'empreinte énergétique du développement et de l'exécution. Le site ne charge que le strict nécessaire, sans ressources superflues.

L'Héritage de Template (Le principe DRY)

Pour maintenir une cohérence absolue, j'ai implémenté un système d'héritage autour d'un fichier maître : base.html.twig.

- **Le Header et le Footer** (issus de mes composants Figma) y sont centralisés.

- Chaque page spécifique (Soins, Sanctuaire, Lexique) vient simplement "injecter" son contenu unique dans cette structure globale.
  Cette méthode respecte le principe **DRY (Don’t Repeat Yourself)** et assure une navigation sans rupture visuelle.

AssetMapper : La modernité sans compilation lourde

Grâce à AssetMapper et au fichier importmap.php, l'application charge les modules JavaScript et le CSS de manière optimisée sans nécessiter de serveur Node.js complexe. C'est une solution performante qui s'intègre parfaitement à l'environnement conteneurisé Docker.



```
<!DOCTYPE html>
<html>
    <head>
        <meta charset="UTF-8">
        <title>{% block title %}L'Empreinte - Shiatsu & MTC{% endblock %}</title>
        <link rel="icon" href="data:image/svg+xml,<svg xmlns=%22http://www.w3.org/2000/svg%22 viewBox=%220 0 128 128%22><text y=%221.2em%22 font-size=%2290%22>

        {{ importmap('app') }}

        {% block stylesheets %}{% endblock %}
        {% block javascripts %}{% endblock %}
    </head>
```

Orchestration des ressources statiques via la fonction Twig importmap, permettant un chargement optimisé des composants sans dépendance à un serveur de compilation lourd.

6.4 Gestion du Design System et Réutilisabilité

Pour que le site soit cohérent, j’ai créé un **Design System** composé de briques réutilisables.

Le Header dynamique : Il change tout seul selon que le visiteur soit un simple curieux, Charles (le patient) ou Sandrine (l'admin).

Les Messages Flash : J'ai soigné les notifications de succès. Elles utilisent une couleur Zen Forest avec un contraste étudié pour rester lisibles par tous (Accessibilité). Un script Stimulus JS gère leur disparition fluide après 5 secondes, respectant ainsi le critère de dynamisme des interfaces sans perturber la navigation.

6.5 Éthique Numérique : Accessibilité et Éco-conception

Un sanctuaire numérique ne peut être complet s'il exclut une partie de ses visiteurs ou s'il s'avère énergivore. J'ai donc intégré deux piliers fondamentaux dans la réalisation de l'interface :

A. Accessibilité (Respect du RGAA)

Mon approche ne s'est pas limitée à l'esthétique "Zen", elle a intégré des normes d'accessibilité pour garantir une expérience inclusive :

Contrastes chromatiques : La palette Zen-Sand (#F7F1E8) et Zen-Ink (#2D2926) a été testée pour garantir un ratio de contraste conforme aux normes WCAG, assurant une lisibilité optimale pour les personnes malvoyantes.

Navigation au clavier : L'utilisation de composants Stimulus JS respecte le focus manager pour permettre une navigation fluide sans souris.

Sémantique HTML5 : L'usage rigoureux des balises <main>, <article>, <nav> et des attributs aria-label sur les boutons de réservation permet aux lecteurs d'écran (type VoiceOver) de restituer fidèlement l'arborescence du site.

B. Éco-conception (Performance et Sobriété)

Dans une démarche de durabilité, j'ai optimisé l'empreinte carbone de l'application :

Suppression du build Node.js : En choisissant **AssetMapper**, j'évite l'installation de milliers de dépendances (node_modules) et les processus de compilation lourds. Le navigateur télécharge uniquement ce dont il a besoin.

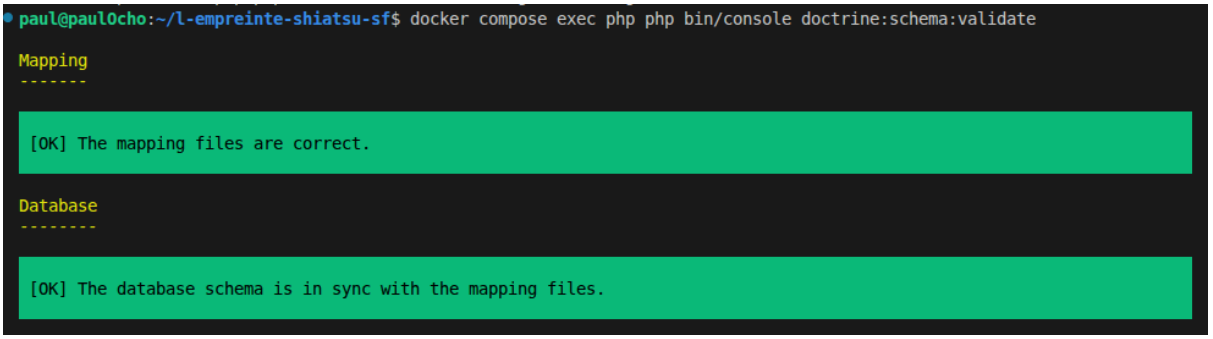
Optimisation des actifs : Les images du sanctuaire passent par un filtre d'optimisation pour réduire leur poids sans sacrifier la qualité visuelle, limitant ainsi la consommation de données (Data) du patient.

Moteur Rust : L'utilisation de **Tailwind CSS v4** (compilé en Rust) permet une génération de styles ultra-rapide, réduisant le temps CPU lors du développement et du déploiement.

VII. RÉALISATION BACK-END

7.1 Architecture, Migrations et Sécurité

7.1.1 Intégrité du Schéma et Mapping ORM



Validation de l'intégrité du schéma relationnel et conformité du mapping ORM.

7.1.2 Anonymisation et Confidentialité (UUID)

```
#[ORM\CustomIdGenerator(class: 'doctrine.uuid_generator')]
```

Au-delà de l'intégrité du schéma, la sécurité du back-end repose sur une gestion fine des accès. L'utilisation d'identifiants UUID garantit l'anonymat dans les URLs, tandis que le contrôle d'accès par rôles assure que seul l'administrateur peut manipuler les données sensibles du catalogue et du lexique.

7.1.3 Sécurité et Authorization (ROLE_ADMIN)

Sur un site professionnel, il est crucial de séparer ce que le public peut voir de ce que la praticienne peut modifier. Pour cela, j'ai mis en place un système de "périmètre sécurisé".

Dans la configuration du site, j'ai défini une règle simple : toutes les adresses internet commençant par /admin sont verrouillées. Seule une personne connectée avec le compte de Sandrine (détenant le rôle Administrateur) peut franchir cette porte numérique. Ce réglage garantit qu'un patient ou un simple visiteur ne pourra jamais accéder aux outils de gestion par erreur.

```
access_control:
    - { path: ^/admin, roles: ROLE_ADMIN }
```

Configuration du verrouillage automatique pour toutes les pages de gestion du cabinet.

Cette approche garantit qu'aucune fonctionnalité de gestion ne peut être exposée par erreur, imposant le rôle ROLE_ADMIN comme clé unique du cockpit de pilotage

7.1.4 Gestion des erreurs de privilèges (Subscriber)

Implémentation d'un écouteur d'événements (*EventSubscriber*) pour intercepter les tentatives d'accès non autorisées et rediriger l'utilisateur vers l'accueil avec un message pédagogique.

```php
use Symfony\Component\HttpFoundation\RequestStack;

class AccessDeniedSubscriber implements EventSubscriberInterface
{
    public function __construct(
        private UrlGeneratorInterface $urlGenerator,
        private RequestStack $requestStack
    ) {}

    public static function getSubscribedEvents(): array
    {
        return [
            KernelEvents::EXCEPTION => ['onKernelException', 255],
        ];
    }

    public function onKernelException(ExceptionEvent $event): void
    {
        $exception = $event->getThrowable();

        if ($exception instanceof AccessDeniedException || $exception instanceof AccessDeniedHttpException) {
            $this->requestStack->getSession()->getFlashBag()->add(
                'error',
                'Votre accès est limité à votre espace personnel.'
            );

            $event->setResponse(new RedirectResponse($this->urlGenerator->generate('app_home')));
        }
    }
}
```

Logique de redirection et gestion événementielle — interception des exceptions de sécurité pour un parcours patient sans rupture.

7.1.5 Le contrôle des informations saisies

Je pars du principe qu'un site professionnel doit être protégé contre les erreurs de saisie, qu'elles soient volontaires ou non. J'ai donc installé des "gardes-fous" directement dans le code du serveur pour ne jamais accorder une confiance aveugle aux données qui arrivent des formulaires.

Concrètement, quand Sandrine ajoute un nouveau soin ou quand Charles écrit un message, le système vérifie chaque information avant de l'autoriser à entrer dans la base de données. Par exemple, j'ai configuré le code pour qu'il soit impossible de valider un soin si le prix est négatif ou si le titre est oublié. Si une donnée est incorrecte, le site bloque l'enregistrement et affiche un message clair à l'utilisateur pour qu'il corrige sa saisie.

Cette rigueur permet de garder une base de données "propre" et évite que le site ne plante à cause d'une information mal remplie. C'est une étape invisible pour l'utilisateur, mais indispensable pour assurer la fiabilité et la sécurité de l'application sur le long terme.

7.1.6 Protection contre les failles majeures (XSS et CSRF)

Pour garantir la sécurité des données de Sandrine et de ses patients, j'ai mis en place des gardes-fous contre les deux attaques les plus courantes du web.

A. Prévention de la faille CSRF (Cross-Site Request Forgery)

Cette attaque consiste à forcer un utilisateur authentifié à envoyer une requête malveillante à son insu. Pour contrer cela, j'utilise le système de **jetons (tokens) CSRF** natif de Symfony.

Fonctionnement : À chaque génération de formulaire (Contact, Réservation, Connexion), Symfony injecte un jeton unique et temporaire.

Validation : Lors de la soumission, le serveur vérifie la validité du jeton. Si le jeton est manquant ou invalide, la requête est rejetée.

```
  <input id="exchange__token" type="hidden" name="exchange[_token]" data-
  controller="csrf-protection" value="csrf-token">
</form>
```

Jeton de sécurité CSRF généré automatiquement par Symfony pour protéger le formulaire.

B. Prévention de la faille XSS (Cross-Site Scripting)

La faille XSS permet à un attaquant d'injecter des scripts malveillants dans les pages vues par d'autres utilisateurs.

Ma solution : J'utilise le moteur de rendu **Twig**. Par défaut, Twig applique un "auto-escaping" (échappement automatique). Cela signifie que tout texte saisi par un utilisateur est neutralisé avant d'être affiché.

Exemple : Si un utilisateur saisit <script>alert('hack')</script>, Twig affichera le texte tel quel à l'écran au lieu de l'exécuter.

C. Validation et Nettoyage des données

En complément, j'utilise les **Contraintes de validation** de Symfony (Assert) dans mes entités. Cela garantit que seules des données au format attendu (ex: un email valide, un prix positif) entrent dans ma base de données PostgreSQL, bloquant ainsi toute tentative d'injection à la racine.

7.2 Logique Métier : L'intelligence du Sanctuaire

La logique métier de "L'EMPREINTE" dépasse la simple gestion de données pour devenir un véritable moteur de soin. Elle automatise la transmission du savoir et sécurise chaque étape du parcours patient, transformant l'application en un prolongement intelligent du cabinet de Sandrine.

7.2.1 La Transmission du Savoir (LexiconLinker)

Je ne voulais pas que Sandrine passe des heures à créer des liens manuels dans ses articles. Mon code agit comme un **assistant invisible** : il lit l'article, reconnaît les mots compliqués du Shiatsu et pose dessus une bulle d'explication. Sandrine écrit son texte comme elle le ferait dans un carnet, et le site s'occupe de la pédagogie tout seul.

La Problématique métier

Sandrine Tribhou utilise des termes techniques (Qi, Méridiens, Hara, etc.) essentiels à la compréhension de sa pratique. Multiplier les notes de bas de page ou forcer l'utilisateur à chercher des définitions manuellement nuirait à l'expérience "Zen" et immersive du Sanctuaire.

Le concept du "Lecteur Attentif"

Le but est que Sandrine puisse écrire ses articles normalement, sans s'occuper de faire des liens. Le système se comporte comme un lecteur expert qui "habille" le texte en temps réel :

Techniquement, j'ai implémenté un service qui scanne les articles pour y injecter du **maillage interne automatique**. Le système identifie les termes techniques et les transforme en **liens hypertextes interactifs**. Au survol, une bulle (Popover) affiche la définition, et au clic, le patient est redirigé vers la fiche complète du lexique. Cela renforce l'autorité sémantique du site tout en éduquant le patient.

```php
foreach ($terms as $lexicon) {
    $term = $lexicon->getTerm();
    $slug = $lexicon->getSlug(); // On récupère le slug pour le lien
    $definition = htmlspecialchars($lexicon->getDefinition());

    // Remplacement par un lien interactif (A) au lieu d'un simple SPAN
    $replacement = sprintf(
        '<a href="/lexique/%s" class="lexicon-trigger group relative border-b border-dotted border-zen-forest">%s<span class="lexicon-bubble">%s</span></a>',
        $slug,
        $term,
        $definition
    );

    $text = preg_replace('/\b' . preg_quote($term, '/') . '\b/i', $replacement, $text);
}
```

Algorithme scannant le texte pour lier dynamiquement les mots techniques aux définitions de la base de données.

Ce qui suit est un exemple pour montrer que lorsque l'on passe la souris sur un mot moins connu, une pop up apparaît avec sa définition.



7.2.2 La Maîtrise du Temps (Algorithme de Réservation)

Un agenda de cabinet, c'est fragile. Si deux personnes réservent en même temps ou si Sandrine n'a pas le temps de ranger sa salle entre deux soins, le service n'est plus "Haute Couture". Mon algorithme interdit physiquement ces erreurs. Il calcule une **"Respiration"** de 15 minutes. Si on essaie de forcer un rendez-vous durant ce battement, le système dit non. C'est la technologie qui protège le calme de la praticienne.

```php
    public function isSlotAvailable(\DateTimeInterface $requestedStart, int $durationMinutes): bool
{
    // On crée un objet DateTime mutable pour éviter les erreurs d'interface
    $start = \DateTime::createFromInterface($requestedStart);

    // Calcul de la fin (Soin + 15 min de respiration)
    $requestedEnd = (clone $start)->modify('+' . ($durationMinutes + 15) . ' minutes');

    // Marge de sécurité de 90 min avant (préparation du cabinet)
    $bufferStart = (clone $start)->modify('-90 minutes');

    $qb = $this->createQueryBuilder('a')
        ->select('COUNT(a.id)')
        ->where('a.dateTime < :requestedEnd')
        ->andWhere('a.dateTime > :bufferStart')
        ->setParameter('requestedEnd', $requestedEnd)
        ->setParameter('bufferStart', $bufferStart);

    return (int) $qb->getQuery()->getSingleScalarResult() === 0;
}
```

*L'algorithme isSlotAvailable sécurise la prise de rendez-vous. Pour éviter le surlignage jaune (lié à l'immuabilité des interfaces de dates), j'utilise DateTime::createFromInterface. La règle métier est stricte : un créneau n'est déclaré "disponible" que si aucun autre soin n'existe dans une fenêtre incluant **90 minutes de préparation en amont** et **15 minutes de respiration en aval**.*

*Note : L'utilisation du mot-clé **clone** est ici indispensable car l'objet DateTime est mutable en PHP. Sans clonage, la modification du paramètre de fin impacterait l'heure de début, faussant l'intégralité du calcul.*

L'algorithme de réservation intègre une règle de confort impérative : un temps de 'Respiration' de 15 minutes est automatiquement ajouté à la fin de chaque prestation. Cette marge technique

garantit la ponctualité de l'accueil et permet une transition sereine entre deux patients, sans chevauchement de créneaux en base de données.

7.2.3 La Sécurisation des données sensibles

La protection de la vie privée des patients est au cœur de l'architecture. Pour garantir qu'aucun mot de passe ne circule ou ne soit stocké en clair, le système utilise l'algorithme de hachage natif de Symfony via l'interface UserPasswordHasherInterface. Cette méthode assure une résistance maximale contre les tentatives de lecture directe des données en base PostgreSQL.

L'anonymisation structurelle par UUID

Au-delà des mots de passe, la sécurité concerne l'identité même des patients. Contrairement à une architecture classique utilisant des identifiants incrémentaux (1, 2, 3...), j'ai implémenté des **UUID (Universally Unique Identifiers)** pour toutes les entités sensibles (User, Appointment, TherapeuticExchange).

Bénéfices :

- **Imprédictibilité :** Il est impossible pour un tiers de deviner l'URL d'un dossier patient ou d'un échange.
- **Sécurité anti-scraping :** L'exposition d'un UUID dans l'URL ne donne aucune information sur le volume de la base de données ou l'ordre de création des comptes.

```php
// 1. CRÉATION DE L'ADMIN (Sandrine)
$admin = new User();
$admin->setEmail('sandrine@l-empreinte.fr');
$admin->setRoles(['ROLE_ADMIN']);
$admin->setFirstName('Sandrine');
$admin->setLastName('Tribhou');
$admin->setWallet(0.00);
$admin->setPassword($this->hasher->hashPassword($admin, 'zen-password'));
$manager->persist($admin);
```

L'interface UserPasswordHasherInterface délègue le choix de l'algorithme à Symfony, qui sélectionne par défaut le plus robuste (Argon2id ou BCrypt selon l'environnement), garantissant une protection maximale contre les tentatives de décryptage par force brute..

7.2.4 Indexation et Recherche Croisée

L'implémentation du moteur de recherche global (US 6) illustre la flexibilité de l'ORM Doctrine. En centralisant les requêtes sur deux sources distinctes (Articles et Lexique), le système offre une porte d'entrée unique vers le savoir du cabinet, renforçant la cohérence du site 'L'Empreinte'.



```php
#[Route('/recherche', name: 'app_search')]
6 references
public function index(Request $request, LexiconRepository $lexiconRepo, ArticleRepository $articleRepo): Response
{
    // On récupère le mot tapé dans l'URL (ex: ?q=qi)
    $query = $request->query->get('q', '');

    $resultsLexicon = [];
    $resultsArticles = [];

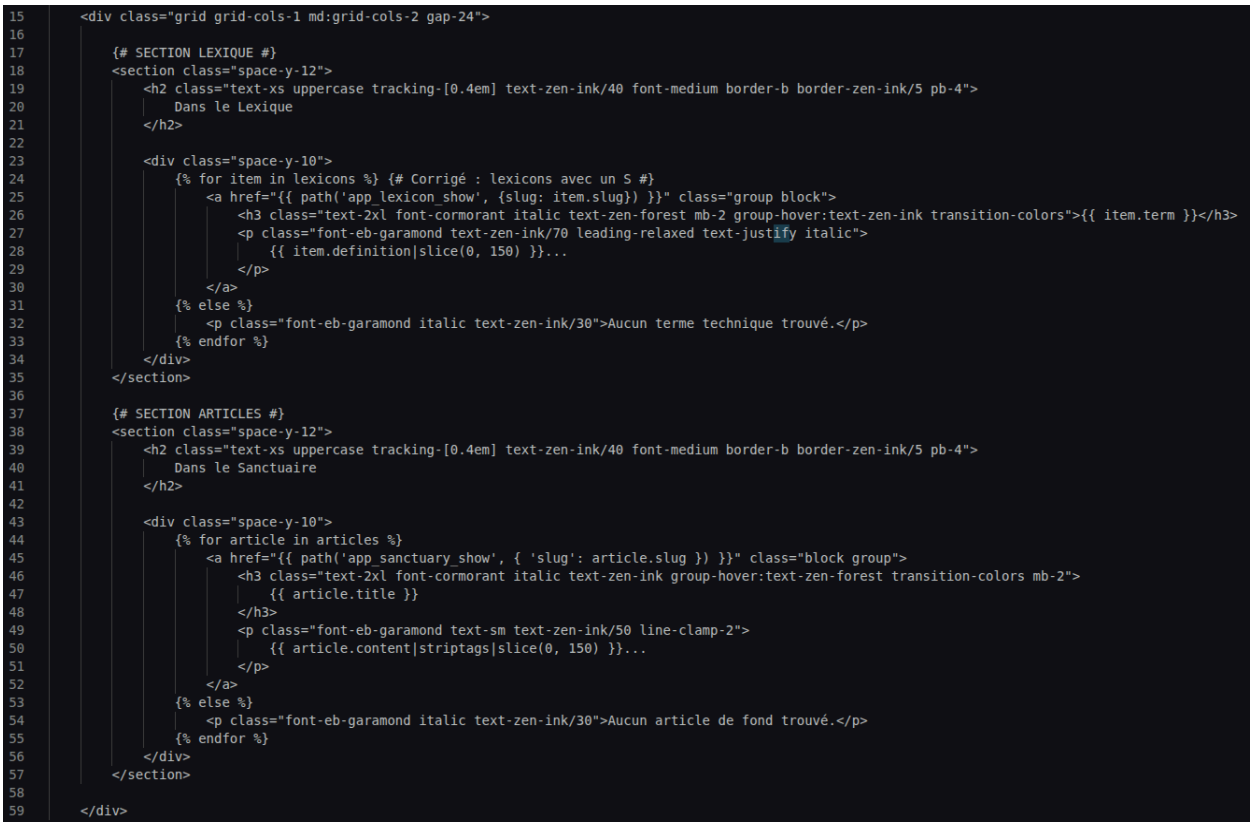
    if (!empty($query)) {
        $resultsLexicon = $lexiconRepo->findBySearch($query);
        $resultsArticles = $articleRepo->findBySearch($query);
    }

    return $this->render('search/index.html.twig', [
        'query' => $query,
            'lexicon' => $resultsLexicon,
        'articles' => $resultsArticles,
    ]);      You, avant-hier • Sprint J-9: Moteur de recherche global, page ar…
}
```

Contrôleur centralisant la recherche sémantique sur plusieurs entités.

Rendu dynamique de la recherche (search/index.html.twig)

Pour que le résultat de la recherche soit à la fois clair et élégant, j'ai utilisé la puissance du moteur de template Twig. Voici l'extrait de code qui gère l'affichage croisé des résultats :



```twig
<div class="grid grid-cols-1 md:grid-cols-2 gap-24">

    {# SECTION LEXIQUE #}
    <section class="space-y-12">
        <h2 class="text-xs uppercase tracking-[0.4em] text-zen-ink/40 font-medium border-b border-zen-ink/5 pb-4">
            Dans le Lexique
        </h2>

        <div class="space-y-10">
            {% for item in lexicons %} {# Corrigé : lexicons avec un S #}
                <a href="{{ path('app_lexicon_show', {slug: item.slug}) }}" class="group block">
                    <h3 class="text-2xl font-cormorant italic text-zen-forest mb-2 group-hover:text-zen-ink transition-colors">{{ item.term }}</h3>
                    <p class="font-eb-garamond text-zen-ink/70 leading-relaxed text-justify italic">
                        {{ item.definition|slice(0, 150) }}...
                    </p>
                </a>
            {% else %}
                <p class="font-eb-garamond italic text-zen-ink/30">Aucun terme technique trouvé.</p>
            {% endfor %}
        </div>
    </section>

    {# SECTION ARTICLES #}
    <section class="space-y-12">
        <h2 class="text-xs uppercase tracking-[0.4em] text-zen-ink/40 font-medium border-b border-zen-ink/5 pb-4">
            Dans le Sanctuaire
        </h2>

        <div class="space-y-10">
            {% for article in articles %}
                <a href="{{ path('app_sanctuary_show', { 'slug': article.slug }) }}" class="block group">
                    <h3 class="text-2xl font-cormorant italic text-zen-ink group-hover:text-zen-forest transition-colors mb-2">
                        {{ article.title }}
                    </h3>
                    <p class="font-eb-garamond text-sm text-zen-ink/50 line-clamp-2">
                        {{ article.content|striptags|slice(0, 150) }}...
                    </p>
                </a>
            {% else %}
                <p class="font-eb-garamond italic text-zen-ink/30">Aucun article de fond trouvé.</p>
            {% endfor %}
        </div>
    </section>

</div>
```

Logique Twig de la recherche croisée permettant l'affichage dynamique des articles et des termes du lexique selon les résultats trouvés.

Pourquoi ce choix technique :

- **Logique conditionnelle avancée :** Plutôt qu'un simple affichage, j'utilise la structure {% for ... %} {% else %}. Cela permet de traiter deux scénarios en un seul bloc : si des résultats existent, ils sont mis en forme ; sinon, un message "Aucun article trouvé" s'affiche automatiquement avec une élégante discrétion.
- **Traitement de la donnée en temps réel :** L'utilisation des filtres |striptags et |slice(0, 150) permet de nettoyer le contenu HTML des articles et de n'afficher qu'un extrait calibré. Cela garantit une interface homogène et lisible, quel que soit le volume de texte saisi en base de données par la praticienne.
- **Sécurité et Protection XSS :** En utilisant Twig pour injecter les variables {{ item.term }} ou {{ article.title }}, je bénéficie du système d'auto-échappement natif. Toute tentative d'injection de code malveillant via les formulaires de recherche est ainsi neutralisée par le moteur de rendu, assurant la protection du navigateur du patient.

Bénéfices techniques :

Découplage : L'utilisation de méthodes findBySearch dans les Repositories respecte le principe de responsabilité unique (SOLID).

UX "One-Click" : Le patient accède indifféremment aux concepts (Lexique) et aux conseils (Sanctuaire) via une seule interface, fluide et instantanée.

7.3 Validation Fonctionnelle (Tests réels)



L'interface ne se contente pas d'informer, elle communique avec le patient. Le message flash s'efface automatiquement après 5 secondes, respectant le calme de la navigation.

*Pour garantir la fluidité de l'expérience, les messages de confirmation utilisent le **FlashBag de Symfony**. Techniquement, l'information est stockée de manière éphémère en session, puis injectée dans le template base.html.twig. Un script **JavaScript (Vanilla)** gère ensuite l'opacité et la suppression du nœud DOM après 5 secondes, évitant ainsi toute pollution visuelle durable.*



Le succès de la réalisation back-end est illustré par l'Espace Patient. Ici, les données de l'utilisateur 'Charles' sont récupérées dynamiquement. On y voit l'historique des soins trié par chronologie ascendante, prouvant que la logique de tri implémentée dans le contrôleur répond au besoin de sérénité et d'organisation du patient.

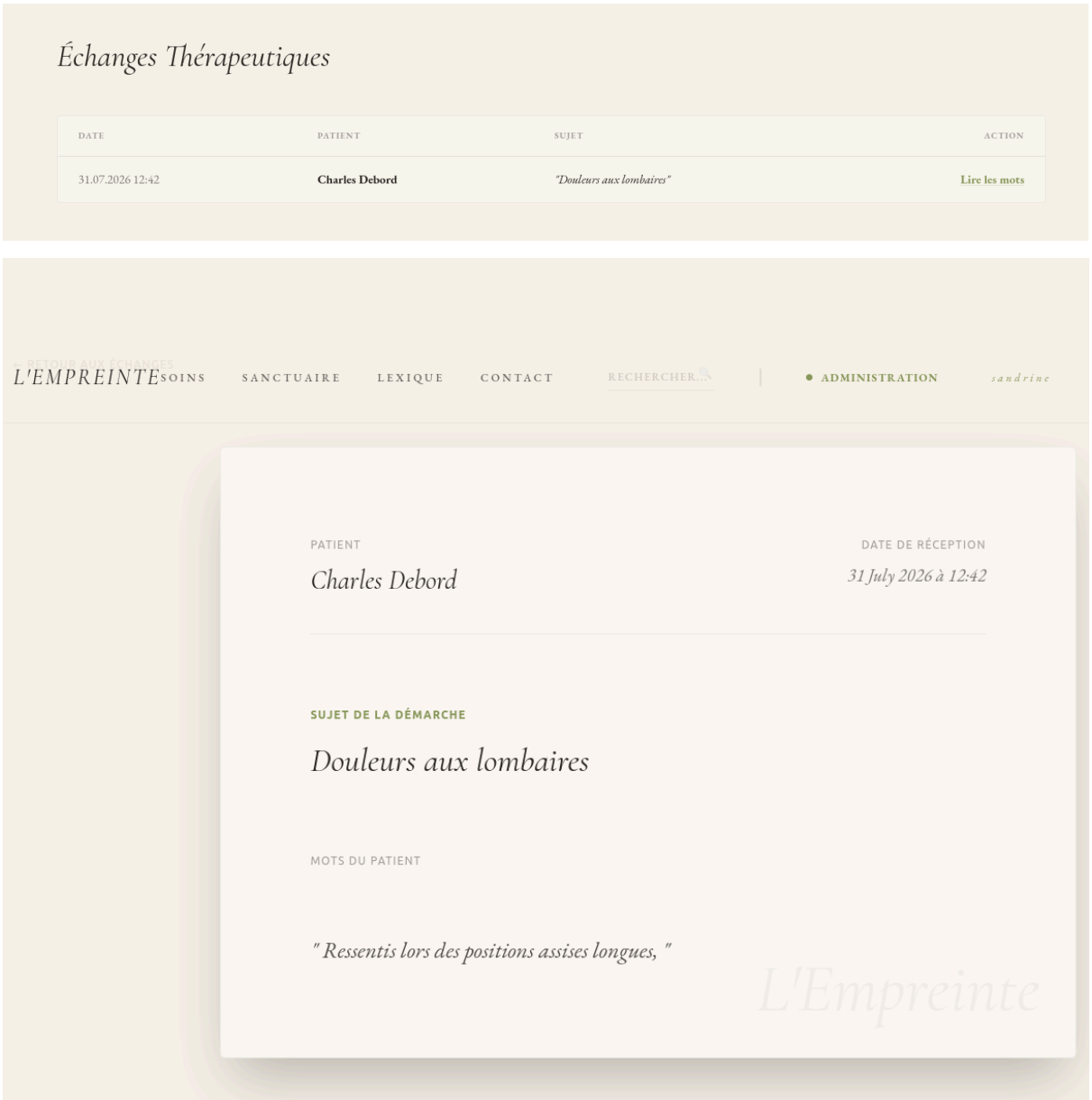
7.4 Gestion de la Relation Patient (CRM)

Le Flux de Synchronicité culmine dans la capacité du site à historiser le dialogue thérapeutique. Cette fonctionnalité permet de transformer un message isolé en un véritable suivi personnalisé. Pour le patient, c'est l'assurance que sa parole est conservée dans un sanctuaire sécurisé ; pour la praticienne, c'est un outil de préparation indispensable avant chaque séance.

7.4.1 Historisation et Déperdition de sens

Techniquement, cette section illustre la récupération dynamique des données liées à l'utilisateur connecté via Doctrine. Chaque "bulle de message" affiche le sujet et un extrait du contenu, triés par date de création. Cette architecture garantit une cohérence totale : l'information saisie dans le formulaire public (Front-Office) est immédiatement restituée dans l'espace privé (Back-Office), sans aucune déperdition de sens.

Par 'déperdition de sens', j'entends la perte du lien thérapeutique originel. En conservant les mots exacts du patient dans son espace sécurisé, le système garantit que l'intention de départ reste intacte et consultable à tout moment, servant de base solide au futur diagnostic de la praticienne.

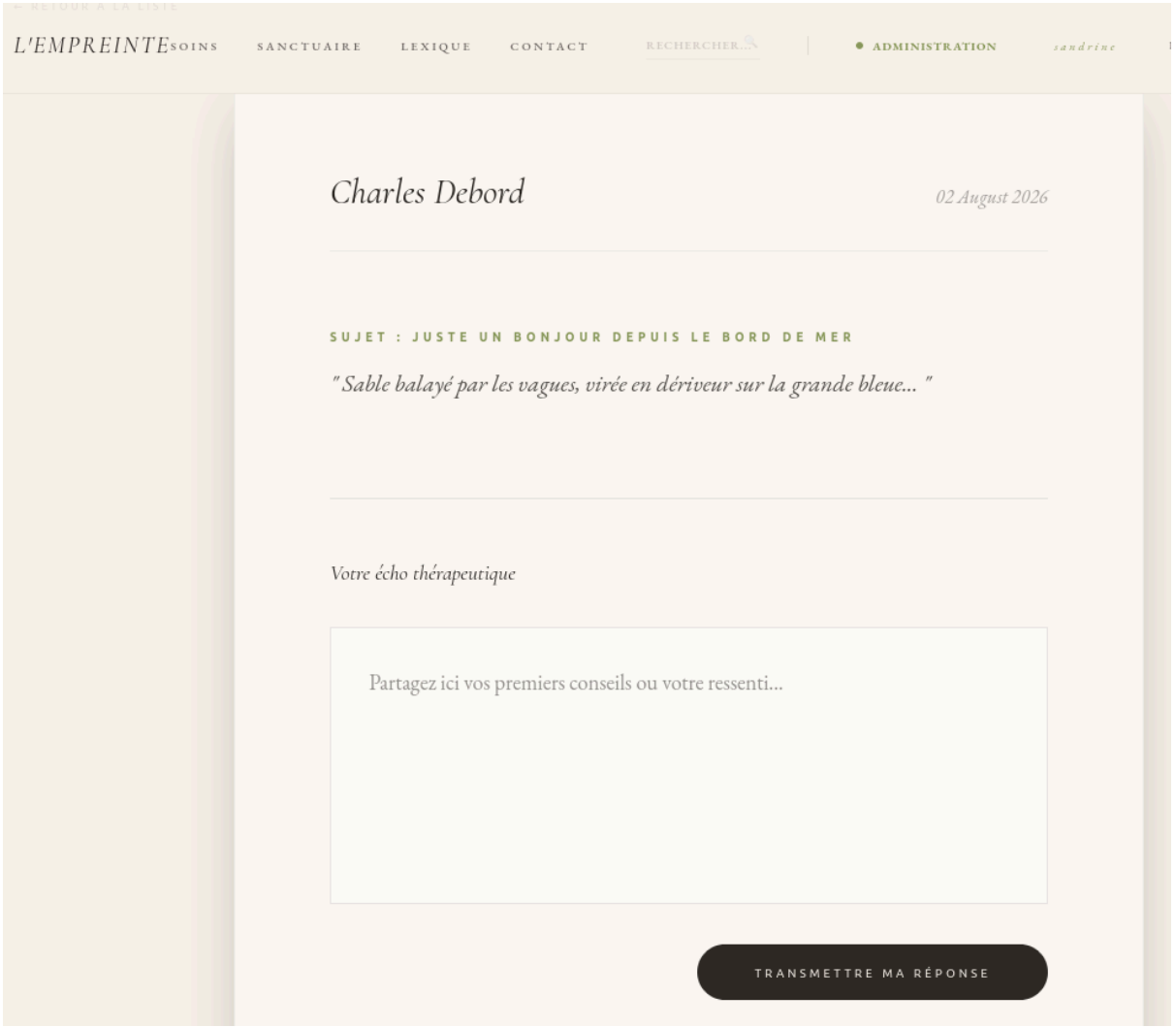


Dualité de l'interface patient. En haut, l'index chronologique des échanges ; en bas, le rendu immersif d'un message spécifique. Cette vue dynamique est alimentée par une relation One-to-Many entre l'utilisateur et ses témoignages en base de données.

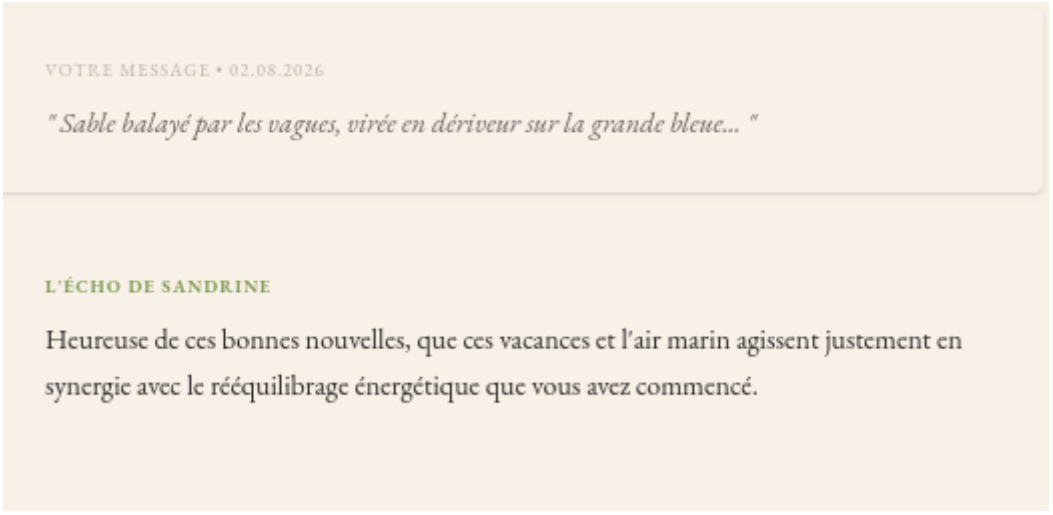
La relation **One-to-Many** entre l'entité User et TherapeuticExchange permet une historisation illimitée. Grâce à l'utilisation de **Lazy Loading (Doctrine)**, le système ne charge les messages que lorsque Charles ou Sandrine ouvrent leur espace respectif, optimisant ainsi les performances et la consommation de mémoire du serveur.

7.4.2 Le Dialogue Bilatéral : L'Écho de la praticienne

Afin de parfaire la relation thérapeutique, l'écosystème permet désormais un échange actif et sécurisé. Cette étape marque le passage d'une simple réception de message à un véritable espace de dialogue dynamique de soin en sa dimension holistique. Chaque réponse déposée par la praticienne devient un jalon essentiel dans le parcours de soin du patient.



Interface de réponse thérapeutique permettant à la praticienne d'initier le dialogue



Restitution de la Synchronicité : le patient reçoit l'écho de la praticienne dans son espace sécurisé.

Pour parfaire ce cycle, j'ai implémenté une fonctionnalité de réponse bidirectionnelle. L'entité TherapeuticExchange a été enrichie d'une propriété practitionerResponse. Désormais, la transmission initiale d'informations se transforme en un véritable dialogue : Sandrine peut apporter un premier éclairage thérapeutique directement depuis son **Espace de Gestion du Soin** (Back-Office), répondant ainsi avec justesse aux maux déposés par le patient. Cette boucle de rétroaction matérialise l'engagement mutuel avant même la rencontre physique.

Au-delà du simple échange, j'ai automatisé le cycle de vie du message. Dès que Sandrine enregistre sa réponse, le système bascule l'échange au statut **'TRAITÉ'** sans qu'elle n'ait besoin de le faire manuellement. Cette règle métier garantit la fiabilité du suivi : si Charles voit son message marqué comme traité, il a la certitude qu'une réponse l'attend. C'est un gain de temps pour la praticienne et une preuve de réactivité pour le patient.

7.5 : L'Économie du Soin — Le Wallet Numérique

Pour fluidifier le parcours de soin de Charles, j'ai implémenté un système de **Wallet numérique**. Ce n'est pas un simple champ d'affichage, mais une brique de gestion financière qui impose des règles de calcul strictes.

La précision chirurgicale du type Decimal

Un développeur doit garantir l'intégrité des données monétaires. J'ai délibérément exclu le type Float (nombre flottant), car il génère des erreurs d'arrondi critiques lors de calculs répétés. J'ai configuré la base de données PostgreSQL avec le type **Decimal(10,2)**. Ce choix technique garantit que chaque centime est stocké comme une valeur exacte, indispensable pour la confiance entre la praticienne et son patient.

Logique métier (Critère AT2 CP7)

Le Wallet est au cœur d'une règle métier : le soin ne peut être validé que si le solde est suffisant ou si le paiement est garanti. En isolant cette logique, je transforme l'application d'un simple site vitrine en un véritable outil transactionnel sécurisé.

7.6 Ma méthode pour vérifier le bon fonctionnement

Pour m'assurer que l'algorithme de réservation est fiable, je ne me suis pas contenté de vérifier que le calendrier s'affichait : j'ai cherché à pousser le système dans ses retranchements en simulant des erreurs humaines courantes (chevauchement, non-respect de la marge de sécurité, solde insuffisant). Le tableau ci-dessous synthétise les cas testés.

Séance testée	Contexte métier	Réponse du système
14h00	Plage horaire libre.	Acceptée
14h05	Tentative de chevauchement sur un soin en cours.	Refusée (Indisponible)
15h10	Demande durant les 15 min de « Respiration ».	Refusée (Zone tampon)
15h15	Réservation juste après le battement de sécurité.	Acceptée
Validation d'un soin	Solde du Wallet insuffisant.	Refusée (message d'erreur affiché)

Cette série de tests confirme que ma fonction isSlotAvailable agit comme une sentinelle, imposant techniquement les règles métiers définies avec Sandrine et garantissant ainsi la sérénité du cabinet.

7.7 Ma démarche de veille : Apprendre et sécuriser

Dans le cadre de ma formation, la veille technologique est un outil d'apprentissage permanent. Pour ce projet, je me suis concentré sur deux aspects pratiques :

A. La découverte de nouveaux outils

J'ai choisi d'utiliser **Symfony 8** et **FrankenPHP**. Ce dernier m'a été conseillé lors de mes lectures techniques car il simplifie beaucoup l'installation avec Docker. Pour moi, c'était l'occasion de découvrir une manière plus moderne et plus directe de faire fonctionner un site PHP, tout en restant dans un cadre compréhensible pour un développeur récent.

B. La sécurité simple et efficace

Ma veille s'est aussi portée sur la sécurité. J'ai appris à utiliser la commande composer audit. C'est un réflexe simple que j'ai pris : elle interroge des bases de données de sécurité pour vérifier si les composants que j'ai installés n'ont pas de failles connues. Si un problème est détecté, je mets à jour la bibliothèque concernée.

C'est une approche humble de la technologie : je reste à l'écoute des bonnes pratiques pour proposer un outil qui soit à la fois moderne et sécurisé pour Sandrine.

7.8 Déploiement et mise en production

Pour la mise en ligne, j'ai utilisé les capacités de **Docker**. L'application est "packagée" dans une image immuable, ce qui garantit qu'elle fonctionnera exactement de la même manière sur mon poste de développement et sur le serveur de production. Le déploiement se fait via une connexion

SSH sécurisée, et la base de données PostgreSQL est protégée par des variables d'environnement non indexées pour garantir la sécurité des accès.

7.8.1 Procédure technique de mise en production

Pour déployer "L'Empreinte" de manière sécurisée et rapide, j'utilise une procédure basée sur Docker. Cela garantit que l'environnement de production est strictement identique à mon environnement de développement.

Les étapes concrètes de déploiement :

1. **Connexion sécurisée au serveur (SSH) :**
   ssh paul@mon-serveur-shiatsu.fr

2. **Mise à jour de l'image de l'application :**
   J'utilise une image Docker pré-construite et immuable. Pour récupérer la dernière version validée du code :
   docker compose pull

3. **Redémarrage des services :**
   Cette commande relance les conteneurs (FrankenPHP et PostgreSQL) sans interruption de service majeure :
   docker compose up -d --remove-orphans

4. **Migration de la base de données :**
   Pour mettre à jour la structure des tables sans perdre les données existantes :
   docker compose exec php php bin/console doctrine:migrations:migrate --no-interaction

Pourquoi cette méthode ?

Cette approche par "conteneurs" évite les erreurs classiques liées aux différences de versions de PHP entre les machines. En une seule ligne de commande, le Sanctuaire est mis à jour, sécurisé et opérationnel.

Documentation et pérennité :

L'intégralité de cette procédure de déploiement, ainsi que les variables d'environnement nécessaires, sont documentées dans un fichier README.md à la racine du projet. Cette démarche garantit que la maintenance du serveur peut être effectuée rapidement, même après une longue période d'interruption, respectant ainsi les standards de qualité professionnelle.

7.9 La Dualité Narrative : Sanctuaire et Blog

Pour que l'expérience numérique soit aussi soignée qu'une séance en cabinet, j'ai structuré la diffusion du contenu en deux espaces distincts. Cette séparation permet à Charles de trouver exactement ce dont il a besoin selon son état d'esprit : se préparer à un soin ou approfondir ses connaissances.

7.9.1 Le Sanctuaire — L'intimité de la pratique

Le Sanctuaire est le prolongement direct de la table de pratique des soins. J'ai conçu cet espace pour accompagner le patient dans sa démarche concrète de soin. On y trouve des articles qui traitent de l'expérience vécue :

La préparation du corps : Quelle tenue porter ? Comment se mettre dans une

disposition d'écoute et de réception ?

Les protocoles spécifiques : Le détail du Shiatsu crânien ou du protocole "Harmonie Digestive".

L'après-soin : Les conseils de Sandrine pour prolonger les bienfaits de la séance une fois rentré chez soi.

Techniquement, chaque page du Sanctuaire est optimisée pour le calme : peu de liens externes, une mise en page aérée et une écriture centrée sur le ressenti du patient.

7.9.2 Le Blog — La sagesse en partage

Le Blog est un espace d'ouverture. Ici, Sandrine partage sa culture de la Médecine Traditionnelle Chinoise (MTC) et du Shiatsu de manière plus large. C'est le lieu de la connaissance théorique :

La Saisonalité : Comment adapter son énergie au passage de l'Automne à l'Hiver ?

Les Fondamentaux : Des articles de fond sur le rôle des méridiens ou l'équilibre du Qi.

Le Shiatsu au quotidien : Des réflexions sur l'impact de la pratique sur le stress moderne ou le sommeil.

Cet espace utilise intensément le **LexiconLinker** que j'ai développé : chaque concept technique cité renvoie vers le dictionnaire, créant un pont permanent entre la lecture d'un article et la compréhension profonde du terme.

7.10 Les Itinéraires de Soin : La réponse aux maux du quotidien

Le catalogue des soins ne doit pas être une simple liste de prix. Pour "L'EMPREINTE", j'ai conçu chaque prestation comme un itinéraire thérapeutique précis. Voici le détail des trois piliers qui structurent l'offre de soins de Sandrine.

7.10.1 La Libération du Mental (Shiatsu Crânien)

Ce soin est une réponse directe au stress de notre époque. Le protocole se concentre sur les zones de tension accumulées dans le haut du dos, la nuque et le crâne. Techniquement, l'application met en avant ce soin pour les patients cherchant à apaiser une charge mentale lourde ou des troubles du sommeil. L'objectif est de recréer un espace de silence intérieur.

7.10.2 L'Ancrage du Hara (Harmonie Digestive)

En MTC, le ventre (le Hara) est le centre des émotions. Ce protocole spécifique vise à libérer les blocages abdominaux. J'ai veillé à ce que la fiche de ce soin explique clairement l'importance du "deuxième cerveau" dans l'équilibre global du corps. C'est un itinéraire souvent recommandé après un changement de saison ou une période de fatigue émotionnelle.

7.10.3 Le Geste Originel (Shiatsu Traditionnel Namikoshi)

C'est le cœur de l'expertise de Sandrine. Ce soin complet du corps suit la lignée japonaise originelle. Il agit sur l'ensemble des méridiens pour rétablir une circulation fluide du Qi. Sur le site, ce soin est présenté comme le point de départ idéal pour tout nouveau patient souhaitant découvrir l'empreinte thérapeutique du cabinet.

7.11 La Dynamique Éditoriale : Le Cycle des Saisons (Espace Blog)

Un site de qualité ne peut pas être une simple vitrine figée. Pour que l'application vive au rythme du cabinet, j'ai développé un **Espace Blog** dont la structure repose sur l'un des piliers de la Médecine Traditionnelle Chinoise : **le cycle des cinq saisons**.

L'enjeu fonctionnel de cette partie du site

L'objectif n'est pas de proposer de la lecture généraliste, mais de créer un outil de fidélisation. Cet espace permet à Sandrine de publier des articles qui conseillent Charles sur l'entretien de son énergie tout au long de l'année. J'ai configuré le système pour que ces articles soient classés par catégories saisonnières, permettant un affichage dynamique : le site peut ainsi mettre en avant les conseils "Hiver" au moment le plus opportun pour le patient.

L'interactivité avec le Lexique

C'est dans le Blog que mon service **LexiconLinker** prend toute sa mesure. Chaque article rédigé par Sandrine est automatiquement scanné par le serveur. Si elle mentionne l'énergie d'un organe ou un méridien spécifique à une saison, le mot se transforme instantanément en lien interactif.

Les cinq piliers du contenu :

Le Printemps (Énergie du Bois) : Articles sur le renouveau et la détoxification.

L'Été (Énergie du Feu) : Conseils sur la circulation et la plénitude.

L'Intersaison (Énergie de la Terre) : Focus sur l'ancrage et la digestion (le centre).

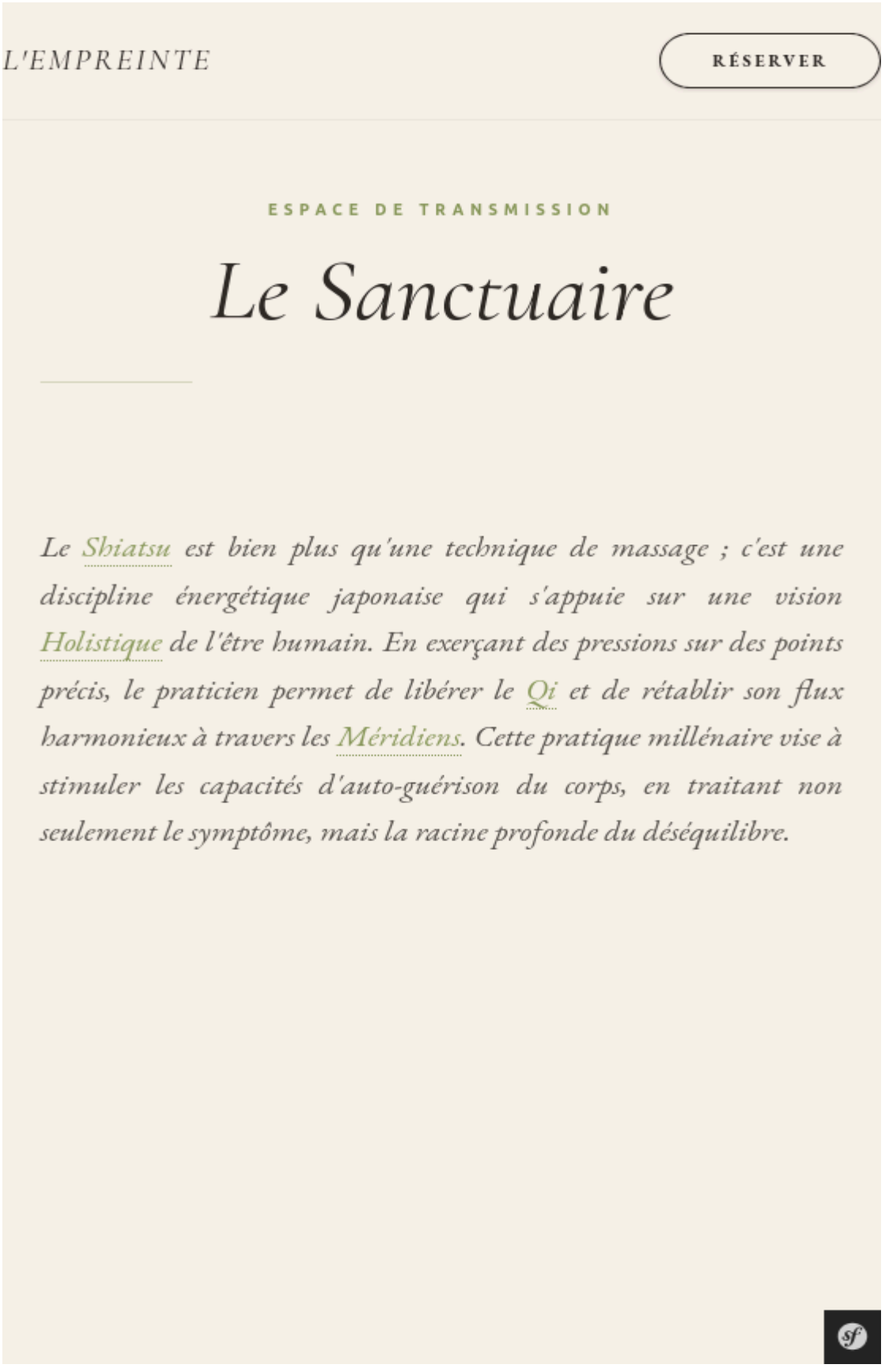
L'Automne (Énergie du Métal) : Travail sur le souffle et l'immunité.

L'Hiver (Énergie de l'Eau) : Articles sur la préservation des réserves et le repos profond.

Cette architecture transforme le blog en une véritable encyclopédie personnalisée, où chaque saison devient une nouvelle occasion pour le patient de se connecter à la pratique de Sandrine.

7.12 L’adaptation mobile : Le Sanctuaire au creux de la main

Pour que l'expérience de "L’EMPREINTE" soit totale, j'ai accordé une importance majeure à l'adaptabilité du site sur tous les supports (Responsive Design). Le bien-être ne doit pas être limité à l'écran d'un ordinateur ; Charles doit pouvoir consulter un conseil saisonnier sur le blog ou réserver une séance d'urgence aussi facilement depuis son téléphone.



Interface mobile de L'Empreinte : adaptation de la structure asymétrique pour une lecture fluide au pouce.

7.12.1 Le défi de la "Chicane" sur petit écran

Mon plus grand défi a été de transposer mon design asymétrique (la Chicane) sur mobile sans perdre l'élégance du projet. Sur un écran de smartphone, l'espace est réduit. J'ai donc configuré le système pour que les blocs qui se croisent sur ordinateur s'empilent avec fluidité sur mobile.

J'ai veillé à conserver ce que j'appelle le "vide sacré" : même sur un petit écran, le texte n'étouffe pas l'image. Les marges restent généreuses pour que la lecture reste un plaisir et non un effort.

7.12.2 Une approche technique "Mobile-First" avec Tailwind v4

Techniquement, j'ai utilisé la puissance de **Tailwind CSS v4** pour gérer ces transitions. J'ai adopté une stratégie "Mobile-First" : le code est d'abord écrit pour le téléphone, puis s'enrichit au fur et à mesure que l'écran s'élargit.

Structure HTML et classes utilitaires (Tailwind CSS)

Pour réaliser l'adaptation mobile, j'ai utilisé les "breakpoints" (points de rupture) de Tailwind CSS. Voici un extrait du code gérant la mise en page :

```
6   <main class="max-w-[1440px] mx-auto px-[150px] pt-32 pb-32 min-h-screen">
7
8       <header class="mb-20">
9           <h1 class="text-5xl font-cormorant italic text-zen-ink">
10              Résultats pour : <span class="text-zen-forest">"{{ query }}"</span>
11          </h1>
12          <div class="h-[0.5px] w-48 bg-zen-forest/30 mt-4"></div>
13      </header>
14
15      <div class="grid grid-cols-1 md:grid-cols-2 gap-24">
```

Interface Mise en œuvre du Responsive Design via les classes utilitaires de Tailwind (gestion des marges et des colonnes).

Arguments techniques sur ce choix :

Approche Mobile-First : Par défaut, j'utilise la classe grid-cols-1 (une seule colonne pour les téléphones). La classe md:grid-cols-2 ne s'active que sur les écrans plus larges. Cela garantit que le site est léger et rapide à charger sur mobile.

Sobriété du code (Maintenance) : Plutôt que de créer des feuilles de style CSS séparées et complexes (Media Queries), tout le comportement responsive est écrit directement dans le HTML. C'est plus facile à maintenir et cela réduit les risques de bugs d'affichage.

Accessibilité Tactile : Comme on le voit dans le code, les marges (gap-24) et les zones de clic sont généreuses. C'est un choix volontaire pour que la navigation reste confortable, même pour des personnes ayant des difficultés de précision motrice.

Points d'attention particuliers :

Le menu "Livre d'Art" : Sur mobile, la navigation se transforme en un menu épuré qui n'apparaît que si on le sollicite, laissant toute la place au contenu.

L'ergonomie tactile : J'ai redimensionné les zones de clic, notamment pour le bouton "Réserver", afin qu'elles soient parfaitement adaptées à une utilisation au pouce.

La légèreté des images : Grâce à AssetMapper et aux classes natives de Tailwind, les images s'adaptent à la taille de l'écran pour ne pas consommer inutilement le forfait data du patient, respectant ainsi ma démarche d'éco-conception.

VIII. BILAN ET ÉVOLUTIONS

8.1 Bilan Personnel et Pédagogique

La réalisation du site "L'Empreinte" a été une expérience formatrice majeure. Elle m'a permis de mettre en pratique l'intégralité du cycle de développement logiciel : du recueil des besoins métiers auprès de Sandrine Tribhou jusqu'à l'implémentation d'une architecture robuste sous Symfony 8. La rigueur imposée par le concept haute qualité m'a poussé à ne jamais sacrifier la performance du code à la facilité technique, notamment par l'usage systématique des UUID et du typage strict en PHP 8.4.

8.2 Évolutions futures du projet

Bien que le MVP (Minimum Viable Product) soit opérationnel et sécurisé, l'application est conçue pour évoluer vers de nouveaux services :

- **Module de statistiques** : Permettre à la praticienne de visualiser la saisonnalité des soins pour adapter ses articles dans le Sanctuaire.
- **Paiement fractionné** : Intégration complète de l'API Stripe pour la gestion automatique des forfaits de 5 ou 10 séances.
- **Anamnèse dématérialisée** : Création d'un formulaire de santé intelligent, dont les données seraient chiffrées en base pour un suivi thérapeutique encore plus fin.
- **Couverture de tests (Qualité logicielle) :** La priorité immédiate après le lancement sera la mise en place d'une suite de tests unitaires automatisés. Cela permettra de sécuriser les parties critiques de l'application, comme l'algorithme de calcul des marges de "Respiration", et de garantir la fiabilité du code sur le long terme.

IX. CONCLUSION

Le projet **"L'EMPREINTE"** a été bien plus qu'un simple exercice de développement ; il a représenté une immersion complète dans la création d'un univers numérique où la technologie s'efface pour se mettre au service de l'humain et du soin.

En choisissant une stack moderne (**Symfony 8, Tailwind v4, PostgreSQL**), j'ai pu garantir à Sandrine Tribhou un outil performant, évolutif et surtout autonome. Ce travail m'a permis de consolider mes compétences techniques, de la rigueur du modèle de données (utilisation des **UUID** pour la confidentialité et du type **Decimal** pour la précision monétaire) à la finesse de l'intégration visuelle asymétrique.

Ce qui distingue fondamentalement "L'EMPREINTE" des solutions web classiques, c'est l'intelligence métier que j'ai pu y injecter. À travers le **LexiconLinker**, j'ai créé une pédagogie invisible qui lie le savoir au texte sans effort pour la praticienne. En automatisant la **"Respiration"** de 15 minutes entre chaque rendez-vous, j'ai fait du code le gardien du calme et de la ponctualité du cabinet.

Aujourd'hui, l'application est bien plus qu'un catalogue : c'est un écosystème vivant. Charles, le patient, y trouve un parcours fluide, de l'échange d'intention initial dans le Sanctuaire jusqu'au règlement sécurisé via son Wallet. Sandrine, quant à elle, dispose d'un système de gestion de contenu (**CMS**) sur-mesure, aussi simple d'utilisation qu'un outil grand public, mais parfaitement adapté aux exigences de la Médecine Traditionnelle Chinoise.

"L'EMPREINTE" est désormais la trace numérique d'un savoir millénaire. Ce projet m'a prouvé que le développement web, lorsqu'il est pratiqué avec rigueur et sensibilité, peut devenir un véritable prolongement thérapeutique, capable de créer de la **Synchronicité** entre le monde virtuel et la rencontre physique en cabinet.

ANNEXE 1 : Carnet de bord des fonctionnalités (User Stories)

Afin de structurer le développement de "L'EMPREINTE", j'ai organisé le travail en **5 Epics** (grands thèmes). Chaque Epic regroupe des **User Stories** précises qui décrivent un besoin utilisateur et son critère de validation. Ce tableau a été mon fil conducteur pour ne rien oublier entre la demande de Sandrine et le code final.

User Stories

Epic	US	Acteur	Intention (Besoin)	Critère de réussite (Validation)
1. Synchronicité	**US 1**	Visiteur	Consulter le catalogue des soins.	Grille tarifaire claire avec durées et bénéfices.
	US 2	Visiteur	Initier un échange d'intention.	Formulaire de contact lié à la création de compte.
	US 3	Patient	Réserver via un calendrier intuitif.	Blocage des créneaux avec 15 min de « Respiration ».
2. Espace Membre	**US 4**	Patient	S'authentifier de manière sécurisée.	Accès protégé, mot de passe haché et UUID.
	US 5	Patient	Suivre son historique de soins.	Liste chronologique des séances passées et à venir.

Epic	US	Acteur	Intention (Besoin)	Critère de réussite (Validation)
	US 6	Patient	Gérer ses forfaits (Wallet).	Affichage du solde exact en temps réel (Decimal).
3. Boutique Cadeau	**US 7**	Client	Acheter une « Carte Expérience ».	Sélection du montant et personnalisation du message.
	US 8	Client	Recevoir la carte par Email.	Génération automatique d'un PDF raffiné.
	US 9	Bénéficiaire	Utiliser son code cadeau.	Déduction automatique du prix lors de la réservation.
4. Pilotage Admin	**US 10**	Sandrine	Accéder au Dashboard de pilotage.	Vue d'ensemble des RDV et messages non traités.
	US 11	Sandrine	Gérer la relation patient (CRM).	Historique complet des échanges par patient.
	US 14	Sandrine	Administrer le Lexique.	Interface CRUD pour enrichir le savoir du site.
5. Gouvernance	**US 15**	Système	Garantir la sécurité (RGPD).	Droit à l'oubli et anonymisation des données.

Matrice de traçabilité des fonctionnalités – synthèse des intentions utilisateurs et critères de validation.

ANNEXE 2 : Sécurité et protection des données (RGPD)

Gérer les informations de Charles ou de Sandrine n'est pas qu'une question de code, c'est une responsabilité. Puisque le site manipule des échanges à caractère thérapeutique, j'ai mis en place une stratégie de protection des données dès la conception du projet.

2.1 La politique de confidentialité par défaut (Privacy by Design)

Au lieu d'utiliser des identifiants simples (1, 2, 3) qui permettraient à un pirate de deviner les adresses des dossiers patients, j'ai généralisé l'usage des **UUID**. Ce sont des codes uniques et complexes qui rendent les liens vers les profils totalement imprédictibles.

2.2 La gestion du "Droit à l'oubli"

Si Charles décide de ne plus fréquenter le cabinet, il peut demander la suppression de son compte. Techniquement, j'ai configuré la base de données avec des relations en "Cascade". Cela signifie que si l'on supprime un patient, le système efface automatiquement et proprement ses rendez-vous et ses messages. Aucune donnée "fantôme" ne reste stockée inutilement sur le serveur.

2.3 Sécurisation des accès et transparence

Sandrine est la seule à avoir accès au pilotage (ROLE_ADMIN). De son côté, Charles ne voit que ce qui le concerne. J'ai également veillé à ce que les mots de passe soient hachés avec les algorithmes les plus récents de Symfony, de sorte que même en cas de fuite de la base de données, les informations personnelles restent indéchiffrables.